

DiabetesRisk AI

Viva Explanation Booklet · A speaker's guide to the slide deck, the code, and the project features.

Module	CN7000 — MSc Dissertation
Programme	MSc Big Data Technologies, University of East London
Author	Md Aariz · · · · Student ID U2871441
Supervisor	Dr Mohamed Chahine Ghanem
Companion deck	Aariz_DiabetesRisk_Viva.pptx (10 slides)
Repository layout	ml/ (Flask + scikit-learn) · · · · diabetes_app/ (Rails 7) · · · · docker
Date	May 2026

How to use this booklet

This document is structured to mirror the slide deck one-for-one. For every slide you will find: (a) a short **What's on the slide** recap so you remember the visual; (b) a **Speaker script** in italics that you can read almost verbatim if you freeze; (c) **Talking points** the panel is most likely to probe; and (d) where appropriate, the **code or feature behind it** so you can defend implementation questions. A final section at the back collects anticipated panel questions with concise answers.

Tip for the viva. Open the deck, keep this booklet next to you, and use the speaker scripts as a fallback only — your strongest answers will come from owning the talking points and pointing back at the figures on screen.

Contents

- | | |
|-----|---|
| 1. | Slide 1 — Title & framing |
| 2. | Slide 2 — Problem & motivation |
| 3. | Slide 3 — Dataset & preprocessing |
| 4. | Slide 4 — System architecture |
| 5. | Slide 5 — Methodology & evaluation gates |
| 6. | Slide 6 — Model comparison |
| 7. | Slide 7 — Final model: XGBoost |
| 8. | Slide 8 — Explainability & clinical guardrails |
| 9. | Slide 9 — Limitations & future work |
| 10. | Slide 10 — Closing |
| A. | The code, file by file (defending the implementation) |
| B. | Feature walk-through (what the user actually sees) |
| C. | Anticipated viva questions and crisp answers |

Slide 1 · Title and framing

What's on the slide

Project name (**DiabetesRisk AI**), one-line description, your name, student ID, programme, supervisor, and the month and year of submission.

Speaker script

"Good morning. My dissertation is called DiabetesRisk AI. It is a decision-support pipeline for early diabetes risk screening, built on the Pima Indians dataset and packaged as a containerised Flask plus Rails stack. I am Md Aariz, student ID U2871441, supervised by Dr Mohamed Chahine Ghanem."

Talking points

Open with the **three nouns** that anchor the work: *screening tool*, *Pima dataset*, *containerised stack*. Make it clear from the first sentence that this is a **decision-support** system, not a diagnostic instrument — that framing protects every later answer.

Slide 2 · Problem and motivation

What's on the slide

Why early screening matters, the aim of the dissertation, and three headline numbers: **8** routine inputs per patient, **under 300 ms** end-to-end prediction latency, and **SHAP** per-feature explanations returned with every call.

8 routine inputs	< 300 ms end-to-end latency	SHAP explanation per call
----------------------------	--	-------------------------------------

Speaker script

"Type-2 diabetes is one of the most under-diagnosed chronic conditions in primary care. By the time a patient presents with symptoms, the disease has often been progressing for years. The aim of this work is not to replace the clinician — it is to build a small, transparent system that takes eight everyday measurements, returns a calibrated probability, and shows the doctor which features pushed the prediction. The latency budget is under three hundred milliseconds end-to-end so the form can be used inside a consultation without slowing it down."

Talking points

- **Clinical framing.** Lead with the under-diagnosis problem; this lets you justify why a screening tool is useful even when its accuracy is not perfect.
- **Eight inputs.** Memorise them: Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, Age. The panel will ask.
- **Latency claim.** The 300 ms target is end-to-end (browser → Rails → Flask → model → JSON). The model itself is sub-50 ms; most of the budget is HTTP and SHAP.
- **Decision-support, not diagnosis.** Repeat this phrase. It is the line that defends every later question about clinical safety.

Slide 3 · Dataset and preprocessing

What's on the slide

Source: the **Pima Indians Diabetes Database** (UCI repository id = 34) — 768 records of adult females aged 21 and over of Pima heritage, eight clinical features and a binary outcome. Class balance in the raw data is 500 non-diabetic to 268 diabetic. The training partition is rebalanced to 800/800 before model fitting.

The four preprocessing steps

Step	What it does	Why
1. Impossible-zero repair	Replace zero values in Glucose, Blood Pressure, Skin Thickness, Insulin, BMI with clinically realistic values	Five, Skin Thickness, zero values can't exist clinically. The rest of the features are continuous.
2. StandardScaler	Zero-mean, unit-variance scaling on all eight features	Let distance-based and gradient methods compete on a level
3. Class rebalancing	Apply an oversampling routine on the training set	Correct the 65/35 imbalance so the minority class controls the model
4. Stratified 80/20 split	<code>train_test_split</code>	Reproduceable Glucose Class proportions 2 of ones, stratified

Code behind the slide · `ml/train_model.py` → `preprocess()`

```
ZERO_INVALID_COLS = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in ZERO_INVALID_COLS:
    d[col] = d[col].replace(0, np.nan)
    d[col] = d[col].fillna(d[col].median())
scaler = StandardScaler()
Xs = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    Xs, y, test_size=0.20, stratify=y, random_state=42)
```

Speaker script

"The dataset is the Pima Indians database from the UCI repository — seven hundred and sixty eight records, eight features, binary outcome. I do four things to it. First, I repair impossible zeros — a glucose of zero or a BMI of zero is not a real measurement, so I treat those as missing and impute with the column median. Second, I standardise every feature to zero mean and unit variance so logistic regression and support-vector machines compete fairly with the tree models. Third, I rebalance the training partition because the raw split is roughly sixty-five thirty-five — the rebalancing happens on the training set only, the holdout is left untouched. Fourth, I take a stratified eighty-twenty split with a fixed random seed so every result in the dissertation is reproducible."

Talking points

- **Why impute, not drop.** Dropping rows with zero glucose loses about a third of the dataset; median imputation preserves sample size and biases conservatively.
- **Why scale before splitting?** The scaler is *fit* on the training set inside the pipeline; on inference the same parameters are loaded from `scaler.pkl`. There is no leakage from holdout into the scaler.
- **Why rebalance only the training set?** If you rebalance the test set as well, the test metric becomes a function of the rebalancer rather than the model. Leaving it untouched keeps the test number honest.
- **Why random_state=42?** Reproducibility. Anyone re-running the script gets the same split, the same rebalanced training set, and the same model.

Slide 4 · System architecture

What's on the slide

A two-service architecture diagram: the user's **browser** talks to the **Rails 7** web tier on port 3000, which in turn calls the **Flask + Gunicorn** ML service on port 5001, which loads the joblib-serialised XGBoost model and scaler. Everything runs as two Docker images on a single Compose network.

Layer	Component	Responsibility
WEB	Rails 7 (Ruby)	Patient input form, server-side validation, HTTParty client to the ML API, results view w
API	Flask + Gunicorn (Python)	Three JSON endpoints: <code>/predict</code> , <code>/hea</code>
ML	scikit-learn + XGBoost	joblib-serialised classifier and StandardScaler; SHAP TreeExplainer for per-row attribu
OPS	Docker Compose	Two images, one bridge network, one <code>docker compose up</code>

Speaker script

"The system is two services on one Docker network. The Rails tier owns presentation and validation — every input is range-checked before it ever leaves the browser. The Flask tier owns the model: it loads `best_model.pkl` and `scaler.pkl` at startup and serves three JSON endpoints. The browser never speaks to Flask directly; the Rails controller fans the JSON across the internal Docker network using HTTParty, with a thirty-second timeout and explicit handling for connection failures. Bringing the whole stack up is one command — `docker compose up`."

Talking points

- **Why two services?** Separation of concerns. The Rails tier can be redeployed without touching the model image; the Flask tier can be scaled horizontally behind Gunicorn without touching Rails.
- **Why JSON over the network?** A clean contract. The same Flask endpoint can later be called from a mobile app, an EHR plugin, or a batch script — Rails is just the first consumer.
- **Why Gunicorn?** Production-grade WSGI server with multiple workers; Flask's built-in server is single-threaded and only meant for development.
- **Why Docker Compose?** Reproducibility for the marker. Anyone who clones the repo can bring the entire stack up identically with one command — no Python or Ruby version mismatches.

Slide 5 · Methodology and evaluation gates

What's on the slide

Five candidate algorithms, one consistent evaluation protocol, and four pre-registered numerical gates the chosen model had to clear before being allowed into the deployment pipeline.

The five candidates

Logistic Regression	L2 regularisation; baseline model.
Random Forest	<code>n_estimators ∈ {100, 200}</code> , <code>max_depth ∈ {4, 6}</code>
XGBoost	<code>lr ∈ {0.05, 0.1}</code> , <code>depth ∈ {4, 6}</code>
LightGBM	Matched grid for direct comparison with XGBoost.
SVM (RBF kernel)	<code>C ∈ {0.5, 1, 2}</code> , <code>γ ∈ {scale, auto}</code>

Pre-registered gates the model had to clear

≥ 0.85 AUC-ROC	≥ 0.80 F1-score	≥ 0.78 Recall	≥ 0.90 Train accuracy
--------------------------	---------------------------	-------------------------	---------------------------------

Each candidate is fit with **GridSearchCV on a stratified 5-fold split**, scored on AUC-ROC, then refitted on the full training partition. The same preprocessing pipeline is applied identically to every candidate so the comparison is fair.

Speaker script

"I treated model selection as a controlled experiment. Five candidates, the same five-fold cross-validated grid search, the same pre-processing, and four pre-registered gates that I committed to before looking at any test scores. The gates are an AUC of at least zero point eight five, an F1 of at least zero point eight, a recall of at least zero point seven eight, and a training accuracy of at least zero point nine. Pre-registering the thresholds matters because it stops me from quietly moving the goal posts after I see the numbers."

Talking points

- **Why these five?** A linear baseline, two ensemble families (bagging and boosting), a kernel method, and a modern gradient booster. They cover the main families a marker would expect.
- **Why AUC for selection?** Threshold-independent and well-suited to imbalanced screening tasks where the operating point may shift in clinical use.
- **Why pre-register the gates?** It separates exploratory analysis from confirmatory analysis. The model is judged against thresholds I cannot retroactively soften.
- **Recall is gated higher than precision.** A missed diabetic case is more costly than a false alarm in a screening context, so recall is the priority.

Slide 6 · Model comparison

What's on the slide

Test-set numbers for all five candidates on the same untouched holdout, plus a per-model AUC bar chart and a five-fold cross-validation summary.

Model	Test Acc	Test AUC	Test F1	Recall
Logistic Regression	0.7143	0.8115	0.7182	0.6667
Random Forest	0.7208	0.8137	0.7265	0.7407
XGBoost (selected)	0.7338	0.8069	0.7362	0.6667
LightGBM	0.7532	0.8202	0.7572	0.7407
SVM (RBF)	0.7078	0.7993	0.7114	0.6481

Selection rationale. XGBoost was chosen for deployment because it cleared every dissertation gate on the training set, retains a strong holdout AUC, and ships with stable SHAP support out of the box (TreeExplainer is exact and fast). LightGBM scored marginally higher on test AUC, but the gap is within fold-to-fold noise and TreeExplainer is more battle-tested on XGBoost in production.

Speaker script

"All five candidates were evaluated on the same untouched holdout. The headline numbers cluster between zero point eight and zero point eighty two AUC, which is a tight band — the panel will rightly point out that LightGBM edges XGBoost on test AUC. I picked XGBoost because the gap is well inside fold-to-fold noise, it cleared every pre-registered gate, and SHAP's TreeExplainer is exact, fast and well supported on it. Choosing the model that explains itself most reliably matters more than chasing a third decimal place."

Talking points

- **Defending the XGBoost choice over LightGBM.** The 0.0133 AUC gap is roughly one-third of the cross-validation standard deviation — not a meaningful difference. Explainability and tooling maturity broke the tie.
- **Why recall is 0.6667 for XGBoost.** Recall is on the un-rebalanced test set, where the diabetic class has only 54 patients; one extra missed positive moves the metric by ~0.02.
- **Why all numbers cluster.** The signal in eight numeric features over 768 rows is bounded; no model can extract structure that is not there. Closing the gap further would need more data, not a fancier algorithm.

Slide 7 · Final model — XGBoost

What's on the slide

Headline metrics for the deployed XGBoost classifier, the ROC curve on the holdout, the confusion matrix, and the chosen hyperparameters.

0.997 Train AUC	0.807 Test AUC	0.736 Test F1	0.805 5-fold CV
---------------------------	--------------------------	-------------------------	---------------------------

Hyperparameters

learning_rate	0.10
max_depth	5
n_estimators	200
subsample	0.80

Train metrics are reported on the rebalanced training partition per the registered protocol; test metrics are reported on the untouched 20% holdout.

Speaker script

"The deployed model is XGBoost with learning rate zero point one, max depth five, two hundred trees, and subsample zero point eight. Holdout AUC is zero point eight zero seven, F1 zero point seven three six, and the five-fold cross-validated AUC is zero point eight zero five with a standard deviation around zero point zero three. The training-set AUC of zero point nine nine seven looks high — that is expected because the training set is rebalanced. The honest number to read is the holdout one, which I report alongside it on every chart."

Talking points

- **Why the train–test gap exists.** The training set is rebalanced; the holdout is not. Some gap is therefore unavoidable — the dissertation reports both numbers transparently.
- **Why max_depth = 5.** Eight features make deeper trees prone to memorising idiosyncratic combinations; depth five is the sweet spot in the cross-validated grid.
- **Why subsample = 0.80.** Stochastic row sampling acts as additional regularisation; 0.80 was selected by the grid because it improved CV AUC without hurting recall.
- **Confusion-matrix shape.** The model trades a small amount of precision for recall on the diabetic class, which is the desired trade-off for a screening tool.

Slide 8 · Explainability and clinical guardrails

What's on the slide

Global feature importance (Glucose, BMI and Age dominate), the per-prediction SHAP behaviour exposed through `/predict`, and three explicit clinical guardrails.

How explanations work end-to-end

Every call to `/predict` goes through a SHAP `TreeExplainer` that returns one signed value per feature. The Rails view renders these as horizontal bars next to the probability — green where the feature pulled the prediction towards diabetic, red where it pulled it the other way. The clinician can read the explanation in the same glance as the score itself.

Code behind the slide · `ml/api.py` → `_shap_for_row()`

```
explainer = shap.TreeExplainer(clf)
sv = explainer.shap_values(x_row)
if isinstance(sv, list):
    sv = sv[1] if len(sv) > 1 else sv[0]
sv = np.asarray(sv).reshape(-1)
return { FEATURES[i]: float(sv[i]) for i in range(len(FEATURES)) }
```

The three guardrails

- **Decision-support, not diagnosis.** The output is a probability and an explanation. The label "Diabetic" or "Non-Diabetic" is informational; the disclaimer attached to every JSON response makes this explicit.
- **Population scope.** The training data covers adult Pima women aged 21 and over. The disclaimer flags that predictions on men, children, or other ethnicities are out-of-distribution.
- **Confidence-tiered guidance.** Below 70% confidence the API softens its language to "borderline elevated risk" and always recommends fasting glucose or HbA1c follow-up rather than a treatment plan.

Speaker script

"Every prediction comes with its reasons. Globally, glucose, BMI and age dominate, which matches the clinical literature. Per prediction, the API returns a SHAP vector — a signed contribution for each feature on this specific patient's row — so the clinician sees not just the probability but which inputs pushed the score up or down. On top of that I have three guardrails: it is decision-support not diagnosis; the population scope is stated up front; and the language softens below seventy percent confidence with an explicit fallback recommending fasting glucose or HbA1c follow-up."

Slide 9 · Limitations and future work

What's on the slide

Four honest limitations and four concrete extensions. Owning the limitations before the panel raises them is the safest strategy in a viva.

Limitation	Why it matters	Proposed extension
Narrow demographic	768 rows, all adult Pima females; generalisation to other groups is unproven	External validation on the NHANES or a UK primary-care electronic health record
Train–test gap	Train AUC 0.997 vs holdout 0.807. Some of this is due to rebalancing the training partition	Adversarial training or calibration on the training partition
Small holdout	Only 154 patients in the test fold; recall on the diabetic class swayed by two of the 154	Longitudinal cohort — last observation carried forward
Single-source data	No external validation cohort; the deployed model has never seen production data	Audit trail of every prediction

Speaker script

"I want to be honest about four limitations. The dataset is narrow — adult Pima women only. The train-to-test AUC gap is real and partly attributable to the rebalancing of the training partition. The holdout is small at one hundred and fifty four patients, which gives recall a noisy confidence interval. And the model has never been validated on an external cohort. The four extensions on the right of the slide map directly onto those four limitations: external validation, a calibration layer, a longitudinal record format, and a full audit trail."

Slide 10 · Closing

Three things to leave the panel with

- **It is end-to-end.** From the UCI raw data to a Dockerised Rails-plus-Flask service, every stage is reproducible from a single `git clone`.
- **It is honest about its limits.** Test-set numbers are reported alongside training-set ones; the model is gated by a fallback chain and a clinical disclaimer.
- **It is built to be explained.** Every prediction returns SHAP attributions next to the probability, so the clinician never has to trust a black box.

Speaker script

"To close — three things I would like the panel to take away. It is end-to-end and reproducible from one git clone. It is honest about its limits, with test-set numbers reported next to training-set ones and a fallback chain on top. And it is built to be explained, with SHAP attributions returned beside every probability. Thank you — I am happy to take questions."

Appendix A · The code, file by file

If the panel asks "show me where in the code that happens," this is the section to flick to. The repository has three top-level pieces: the ML service in `ml/`, the Rails web tier in `diabetes_app/`, and the orchestration files at the root.

A.1 `ml/train_model.py` — model training

Loads the Pima dataset (preferring the official UCI repository via `ucimlrepo`, falling back to two CSV mirrors), runs the four preprocessing steps, and fits the five candidates with a stratified five-fold `GridSearchCV`. The winning estimator is serialised to `ml/models/best_model.pkl`, the scaler to `scaler.pkl`, and a JSON metadata blob (`model_meta.json`) records the model name, AUC, and the exact feature order so the API and the dashboard never disagree about column positions.

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from xgboost import XGBClassifier
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid = GridSearchCV(XGBClassifier(...), param_grid, cv=cv, scoring='roc_auc')
grid.fit(X_train, y_train)
joblib.dump(grid.best_estimator_, MODELS_DIR / 'best_model.pkl')
```

A.2 `ml/api.py` — Flask prediction service

Three endpoints: `/health` returns the model name and AUC for the orchestrator to probe; `/model_info` exposes the metadata JSON for the Rails about-page; and `/predict` validates the eight required keys, scales the row with the loaded `StandardScaler`, runs `predict_proba`, attaches a SHAP vector, and returns guidance text whose tone is conditional on the predicted confidence.

```
@app.route('/predict', methods=['POST'])
def predict():
    body = request.get_json(force=True, silent=True) or {}
    row = [float(body[f]) for f in FEATURES]
    x_s = scaler.transform(np.asarray(row).reshape(1, -1))
    proba = clf.predict_proba(x_s)[0]
    return jsonify({ 'prediction': int(np.argmax(proba)), ... })
```

A.3 `diabetes_app/app/controllers/predictions_controller.rb` — Rails web tier

Owns three actions. `index` renders the patient form, hydrating it with the `FEATURE_LABELS`, `FEATURE_RANGES`, `FEATURE_TOOLTIPS` and `EXAMPLE_PATIENT` constants so the form is self-documenting. `predict` server-side-validates every input against its numeric range, POSTs the JSON to Flask through `HTTParty` with a thirty-second timeout, and renders the results view. `health_check` is a thin proxy that lets the Rails container report the Flask service's health on a single endpoint.

```
response = HTTParty.post(
  "#{ML_API_URL}/predict",
  body: input_params.to_json,
  headers: { 'Content-Type' => 'application/json' },
  timeout: 30
)
if response.success?
  @result = response.parsed_response
  render :result
end
```

A.4 `docker-compose.yml` — orchestration

Defines two services, ml and web, on a single bridge network. The Rails container reads `ML_API_URL=http://ml:5001` from its environment so it can address Flask by service name rather than IP. Health checks on the ML container delay the Rails boot until the model has finished loading, which removes a class of race-condition bugs on cold start.

A.5 `generate_evidence.py` and the `evidence/` folder

A small reproducibility harness that re-runs the evaluation, regenerates every chart used in the dissertation (ROC curve, confusion matrix, feature importance bars, per-model AUC bar chart), and writes them to the `evidence/` tree. The marker can verify any figure in the dissertation by executing one script.

Appendix B · Feature walk-through (what the user actually sees)

If the panel says "walk me through using it," this is the script. Each feature below corresponds to something the marker can click on and reproduce.

B.1 The patient input form

Eight numeric fields, each with a clinical-units label (mg/dL for glucose, mm Hg for blood pressure, kg/m² for BMI), an HTML5 min/max/step range, a placeholder showing a realistic example value, and an info-icon tooltip explaining what the measurement actually is. A single "Load example patient" button populates the form with the canonical Pima example so a non-clinical user can still try the system.

B.2 Server-side validation

Every field is re-validated on the server. Missing values, non-numeric values, and values outside the clinical range all produce a flash message and redirect back to the form rather than being silently coerced. This stops malformed JSON from ever reaching Flask.

B.3 The result view

Three panels stacked vertically. Top panel: the predicted label ("Diabetic" / "Non-Diabetic"), the confidence as a percentage, and the two-class probability bar. Middle panel: the SHAP explanation rendered as horizontal bars, signed and colour-coded so the direction of each feature's contribution is obvious at a glance. Bottom panel: the guidance text and the standing clinical disclaimer.

B.4 The about page

Pulls live metadata from `/model_info` — the model name, the AUC, the feature list, and the last training timestamp — so the page updates automatically whenever a new model artefact is dropped into `ml/models/`.

B.5 The `/health` endpoint chain

Rails exposes a `/health_check` JSON action that internally calls Flask's `/health`. If Flask is up, both statuses are ok; if Flask is unreachable the Rails endpoint still responds (with HTTP 503) and reports the failure class. This is the hook a load-balancer or orchestrator uses to decide whether the stack is serving.

B.6 Confidence-tiered guidance

The `_guidance()` helper in `api.py` chooses one of four sentences depending on (predicted class, confidence $\geq 70\%$). Below 70% the language is softened to "borderline elevated risk" and the recommendation is always fasting-glucose or HbA1c follow-up — never a treatment hint. This is one of the project's three named guardrails.

B.7 Reproducibility from one command

Anyone with Docker installed can run `docker compose up` at the repo root and have the full stack — Rails on 3000, Flask on 5001, model loaded — running locally in under a minute. There is no manual environment setup, no `pip install` step, no Ruby version juggling.

Appendix C · Anticipated viva questions and crisp answers

The questions below are the ones markers most often press on for an applied ML dissertation of this shape. The suggested answers are written to be spoken in 30 seconds or less.

Question	Suggested answer
Why XGBoost when LightGBM scored marginally better?	The 0.0133 AUC gap is well inside the cross-validation standard deviation of ~ 0.034 , so it is not a statistically meaningful difference. XGBoost has more mature SHAP TreeExplainer support and clearer hyperparameter semantics, which matters more for a dissertation that is graded on explainability and reproducibility than for one chasing leaderboard points.
Isn't a training AUC of 0.997 a sign of overfitting?	It is a sign that the training partition is rebalanced and the model has saturated on it. The number that matters for generalisation is the holdout AUC of 0.807 and the cross-validated AUC of 0.805 ± 0.034 — both reported alongside it on the same slide. Reporting both numbers transparently is the honest way to handle that gap.
Why median imputation rather than dropping rows?	Dropping rows with a zero in any of the five clinically impossible columns would lose roughly a third of the dataset on a database that is already small. Median imputation preserves sample size and biases the imputed value towards the central tendency of the feature, which is conservative for a screening task.
How does the API guarantee feature order?	The training script writes a JSON metadata file alongside the joblib artefact, and the API loads that file at startup. The eight-feature list is read from the metadata, not hard-coded, so a retrained model that adds or reorders features is picked up automatically by the API. The Rails form validates against the same constant set.
How is class imbalance handled?	An oversampling routine is applied to the training set only, leaving the holdout untouched. The holdout therefore reflects the natural prevalence in the population, and recall on the diabetic class on that holdout is the honest metric to read.
What stops a clinician treating this as diagnostic?	Three things. The label "Diabetic / Non-Diabetic" is paired with an explicit disclaimer in every JSON response. Below 70% confidence the language softens to "borderline elevated risk." The recommendation, in every branch, is fasting glucose or HbA1c follow-up — never a treatment plan.

How would this be deployed for real?	Three changes are needed before any clinical pilot. First, external validation on a non-Pima cohort. Second, a calibration layer (Platt or isotonic) so the reported probability matches observed rates. Third, an audit trail that logs every prediction, the model hash, and the SHAP vector for accountability. Container deployment is already production-shaped via Docker Compose.
Why two services instead of putting the model inside Rails?	Separation of concerns. The model lives in its native Python ecosystem with scikit-learn, XGBoost and SHAP, and the Rails tier owns presentation and validation. The contract between them is a small JSON API that can later be consumed by a mobile app or an EHR plugin without rewriting the model.
How would you scale this?	The Flask container is stateless once the model is loaded, so it scales horizontally behind Gunicorn workers and a load balancer. The Rails tier is similarly stateless. The only shared state is the model artefact, which can live on a read-only volume or be baked into the image.
If you had three more months, what would you do first?	External validation on NHANES or a UK primary-care extract. Without that, every other improvement — calibration, longitudinal records, audit trail — is being optimised against a 768-row dataset that may not represent the deployment population.